

---

MODULE *downloader*

---

The download pipeline abstraction.

The minimal object both *v2* sync models are about: a set of verified blocks and a set of blocks in flight. A block is requested (enters flight), delivered (leaves flight into verified), or requeued (leaves flight with nothing — timeout, refusal, truncation, not-found, a closed connection); the chain can also be extended directly (mining). Verified blocks are a contiguous prefix of the chain and are never lost.

*protocol.tla* (the concrete stream-level protocol) and *sync\_scheduler.tla* (the peer-choosing scheduler) both refine this module — see *refinement.tla* and *sched\_refinement.tla*. The refinement is over safety only: it ties the pipeline structure of the two models together, saying nothing about fairness or liveness, which each model states in its own terms. In particular the buggy scheduler configurations still refine this module — the stall bugs are liveness bugs, invisible at this altitude.

EXTENDS *Naturals*, *FiniteSets*

CONSTANT *MaxBlock*      the chain is  $1 \dots \text{MaxBlock}$

*Blocks*  $\triangleq 1 \dots \text{MaxBlock}$

VARIABLES

*verified*,      blocks held and verified: always a prefix  $1 \dots k$   
*inflight*      blocks currently being downloaded

*vars*  $\triangleq \langle \text{verified}, \text{inflight} \rangle$

---

Download may begin anywhere in the chain: the pipeline may already hold a verified prefix (a node is born with at least the genesis block, or with nothing when the scheduler starts from scratch).

*Init*  $\triangleq$   
 $\wedge \exists k \in 0 \dots \text{MaxBlock} : \text{verified} = 1 \dots k$   
 $\wedge \text{inflight} = \{\}$

A block neither held nor in flight is requested.

*Request*  $\triangleq$   
 $\exists b \in \text{Blocks} \setminus (\text{verified} \cup \text{inflight}) :$   
 $\wedge \text{inflight}' = \text{inflight} \cup \{b\}$   
 $\wedge \text{UNCHANGED } \text{verified}$

An in-flight block is delivered and verified.

*Deliver*  $\triangleq$   
 $\exists b \in \text{inflight} :$   
 $\wedge \text{verified}' = \text{verified} \cup \{b\}$   
 $\wedge \text{inflight}' = \text{inflight} \setminus \{b\}$

An in-flight block comes back with nothing: timeout, refusal, truncation, not-found, or the connection carrying it closed.

*Requeue*  $\triangleq$   
 $\exists b \in \text{inflight} :$

$$\begin{aligned} & \wedge \textit{inflight}' = \textit{inflight} \setminus \{b\} \\ & \wedge \text{UNCHANGED } \textit{verified} \end{aligned}$$

The chain is extended without a download (mining at the tip).

$$\begin{aligned} \textit{Extend} & \triangleq \\ & \exists b \in \textit{Blocks} \setminus (\textit{verified} \cup \textit{inflight}) : \\ & \quad \wedge \textit{verified}' = \textit{verified} \cup \{b\} \\ & \quad \wedge \text{UNCHANGED } \textit{inflight} \end{aligned}$$

---


$$\textit{Next} \triangleq \textit{Request} \vee \textit{Deliver} \vee \textit{Requeue} \vee \textit{Extend}$$

$$\textit{Spec} \triangleq \textit{Init} \wedge \Box[\textit{Next}]_{\textit{vars}}$$


---

$$\begin{aligned} \textit{TypeOK} & \triangleq \\ & \wedge \textit{verified} \subseteq \textit{Blocks} \\ & \wedge \textit{inflight} \subseteq \textit{Blocks} \end{aligned}$$

A block is never both held and in flight.

$$\textit{Disjoint} \triangleq \textit{verified} \cap \textit{inflight} = \{\}$$


---